CHALLENGE SHEET

Sealdesk

Recover the access code from an encrypted delivery receipt to open a protected document.

DIFFICULTY	TRACK	HOST PORT	LAB ID
MEDIUM	CRYPTO	8080	sealdesk

EASY · **MEDIUM** · HARD · INSANE

VERIFY THE SOLVE

```
python3 ../scripts/check.py sealdesk
```

Brief

SealDesk is a **medium Crypto** lab. The case file contains the plan's encrypted delivery receipt, while the signed document stays held at the records desk until someone supplies its access code. Your goal is to recover that access code, open the document, and submit the returned flag to the checker.

The scenario, people, case files, and records are fictional. Use only the supplied local lab instance.

Setup

Prerequisites:

- Docker Desktop using Linux containers, or Docker Engine on Linux, with Docker Compose `v2.17.0` or later.
- Python 3.10 or later on the host for hash synchronization, reset, and the local checker; no additional Python packages are required.
- A browser or an HTTP client, and an available local port `8080`.
- The first build requires internet access to download the pinned base image and hash-locked Python packages. The running lab needs no external service.

Run the commands from this lab directory. Substitute `python3` for `python` if needed, for example on Linux and macOS.

```
docker compose up -d --build --wait
python scripts/sync_flag_hash.py
docker compose ps
```

Open the local pages:

- Workbench: `http://127.0.0.1:8080/`
- Health check: `http://127.0.0.1:8080/health`

Read the player guide for the lab workflow.

The only published host port is `127.0.0.1:8080`. The application container is not published directly.

To reset the instance and refresh the checker digest:

```
python reset.py
```

The reset rotates the keys, access code, receipt, and flag. Any receipt, code, or flag captured before the reset is no longer valid. The reset does not pull images or rebuild.

To stop the lab and remove its containers and networks:

```
docker compose down
```

The lab keeps its runtime state in memory. Stopping, restarting, or removing the application container creates a fresh instance with a new flag when it starts again. Run `python scripts/sync_flag_hash.py` after every start, or the checker will reject the new flag. The local images remain available for the next start.

Goal

Recover the access code for the case document, open the document through the records desk, and capture the final flag in the form `duck{...}`. Check it from this directory:

```
python ../../../scripts/check.py .
```

The checker prompts for a flag and prints `solved` or `not solved`. It does not repeat the submitted value, and it compares only against the SHA-256 digest in `lab.yml`.

Notes

- This lab is intentionally vulnerable. Do not apply the technique to systems you do not own or have permission to test.
- Treat the lab as a black box: only the workbench and HTTP API at `127.0.0.1:8080` are in scope. The lab directory also holds the source code and maintainer documentation, including the official solution; reading them spoils the challenge.
- The host, Docker, container internals, images, and reset controls belong to the operator and are not challenge assets. In hosted delivery, only the assigned HTTP(S) endpoint is in scope.